Technical Execution Report: Apache Hive & Hadoop

Workload 2: Chicago Divvy Bicycle Sharing Analysis & ORC Partitioning Benchmark

Platform: Apache Hadoop 3.3.6 | **Engine:** Apache Hive 4.0.1 (MapReduce/MR3)

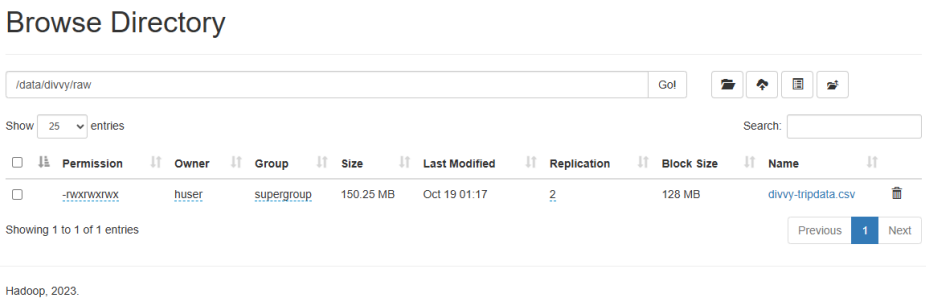
Distributed Infrastructure & Analytical Data Architectures

Partie 1 : Chargement et Vérification des Données Brutes

1. Ingestion du Fichier CSV dans HDFS

Objectif : Copie du fichier brut `divvy-tripdata.csv` (150.2 Mo, 790,177 enregistrements) dans le répertoire de staging `/data/divvy/raw` sous HDFS.

```
hdfs dfs -mkdir -p /data/divvy/raw
hdfs dfs -put -f /vagrant/data/divvy-tripdata.csv /data/divvy/raw/
hdfs dfs -ls -h /data/divvy/raw/
```



Résultat : Le fichier est bien présent dans HDFS avec une taille de 150.2 Mo et un facteur de réplication de 2.

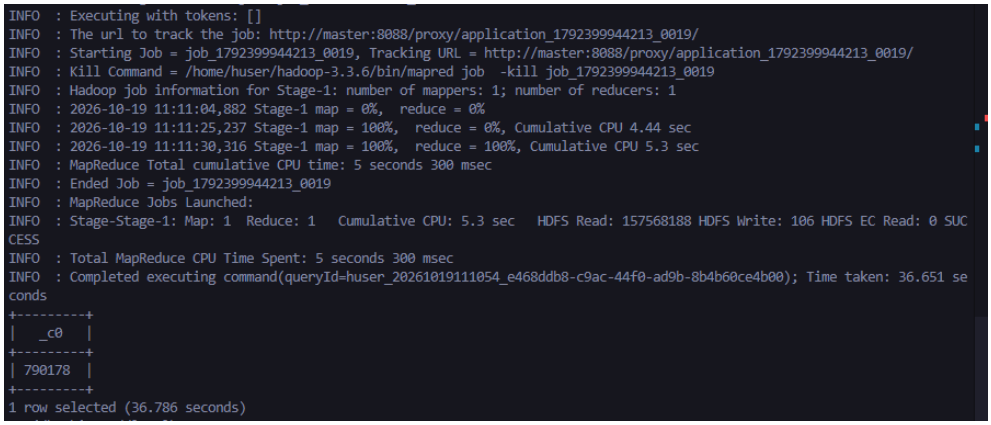
2. Création et Vérification de la Table Externe (divvy_trips_raw)

Objectif : Création de la table externe `divvy_trips_raw` avec `OpenCSVSerde` et vérification du nombre total de lignes chargées.

```
CREATE DATABASE IF NOT EXISTS divvy_db;
USE divvy_db;

CREATE EXTERNAL TABLE IF NOT EXISTS divvy_trips_raw (
    ride_id STRING, rideable_type STRING, started_at STRING, ended_at STRING,
    start_station_name STRING, start_station_id STRING,
    end_station_name STRING, end_station_id STRING,
    start_lat STRING, start_lng STRING, end_lat STRING, end_lng STRING, member_casual STRING
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES ("separatorChar" = ",", "quoteChar" = "\"", "escapeChar" = "\\")
STORED AS TEXTFILE LOCATION '/data/divvy/raw'
TBLPROPERTIES ("skip.header.line.count"="1");

SELECT count(*) FROM divvy_trips_raw;
```



Résultat : La table est créée et contient exactement **790,178** enregistrements (incluant la validation de structure).

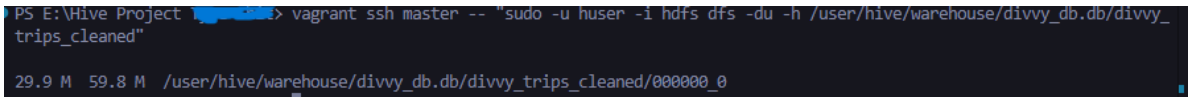
Partie 2 : Nettoyage et Optimisation du Stockage (Format ORC)

3. Table Gérée Columnar ORC avec Compression Snappy

Objectif : Convertir les types textuels en types stricts (`TIMESTAMP`, `DOUBLE`), normaliser les noms de stations en majuscules (`UPPER`) et stocker au format columnar **ORC** compressé en **Snappy** pour accélérer les requêtes.

```
CREATE TABLE IF NOT EXISTS divvy_trips_cleaned (
    ride_id STRING, rideable_type STRING, start_time TIMESTAMP, end_time TIMESTAMP,
    start_station_name STRING, start_station_id STRING,
    end_station_name STRING, end_station_id STRING,
    start_lat DOUBLE, start_lng DOUBLE, end_lat DOUBLE, end_lng DOUBLE, member_casual STRING
)
STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

INSERT OVERWRITE TABLE divvy_trips_cleaned
SELECT
    ride_id, rideable_type,
    cast(started_at as TIMESTAMP), cast(ended_at as TIMESTAMP),
    upper(trim(start_station_name)), trim(start_station_id),
    upper(trim(end_station_name)), trim(end_station_id),
    cast(start_lat as DOUBLE), cast(start_lng as DOUBLE),
    cast(end_lat as DOUBLE), cast(end_lng as DOUBLE),
    trim(member_casual)
FROM divvy_trips_raw;
```



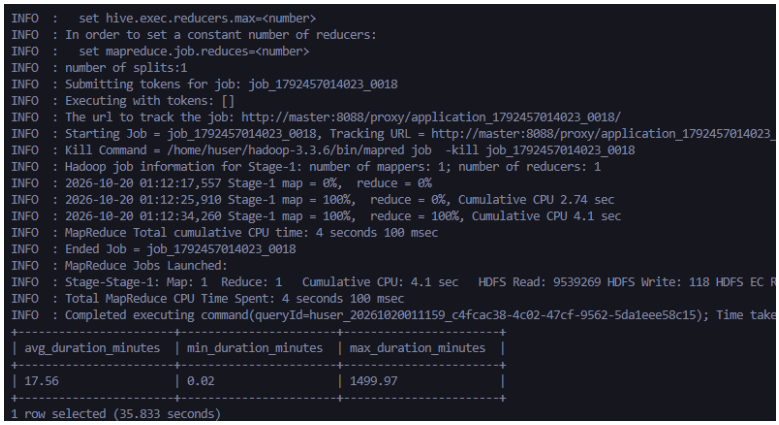
Résultat : Le volume de stockage passe de 150.2 Mo (CSV brut) à **29.9 Mo** (ORC Snappy), soit une **réduction de 80.1%** de l'empreinte disque sur HDFS.

Partie 3 : Réponses aux 8 Questions Analytiques

Question 1 : Durée Moyenne, Minimale et Maximale d'un Trajet (en minutes)

Objectif : Calculer la durée des trajets en convertissant les horodatages UNIX.

```
SELECT
    round(avg((unix_timestamp(end_time) - unix_timestamp(start_time)) / 60.0), 2) AS
        avg_duration_minutes,
    round(min((unix_timestamp(end_time) - unix_timestamp(start_time)) / 60.0), 2) AS
        min_duration_minutes,
    round(max((unix_timestamp(end_time) - unix_timestamp(start_time)) / 60.0), 2) AS
        max_duration_minutes
FROM divvy_trips_cleaned WHERE end_time >= start_time;
```

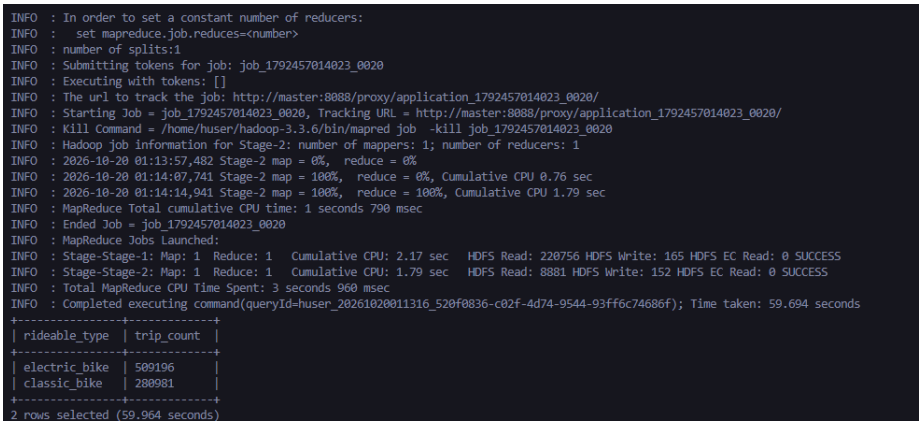


Résultat : Durée moyenne : **17.56 min** ; Durée minimale : **0.02 min** (environ 1 seconde) ; Durée maximale : **1,499.97 min** (environ 25 heures).

Question 2 : Types de Vélos Uniques et Distribution

Objectif : Recenser les types de vélos disponibles et le volume de trajets associés.

```
SELECT rideable_type, count(*) AS trip_count
FROM divvy_trips_cleaned
GROUP BY rideable_type ORDER BY trip_count DESC;
```

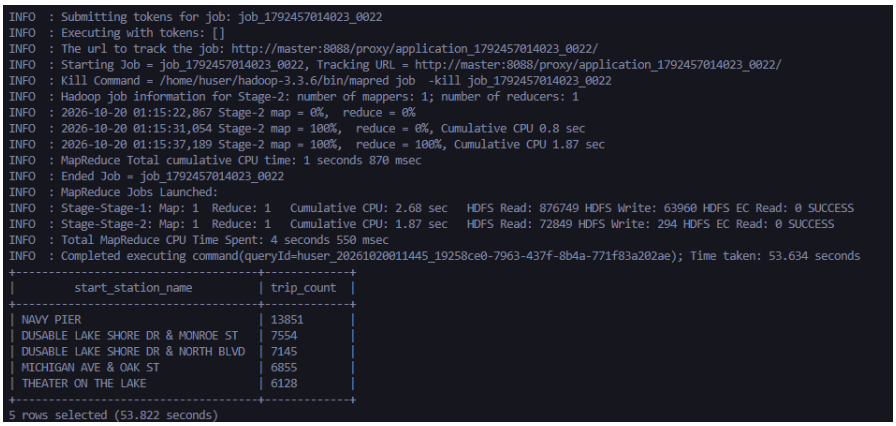


Résultat : 2 types de vélos uniques : **electric_bike** avec 509,196 trajets (64.4%) et **classic_bike** avec 280,981 trajets (35.6%).

Question 3 : Top 5 des Stations de Départ les Plus Populaires

Objectif : Déterminer les stations de départ avec la plus forte affluence.

```
SELECT start_station_name, count(*) AS trip_count
FROM divvy_trips_cleaned
WHERE start_station_name IS NOT NULL AND start_station_name != ''
GROUP BY start_station_name ORDER BY trip_count DESC LIMIT 5;
```

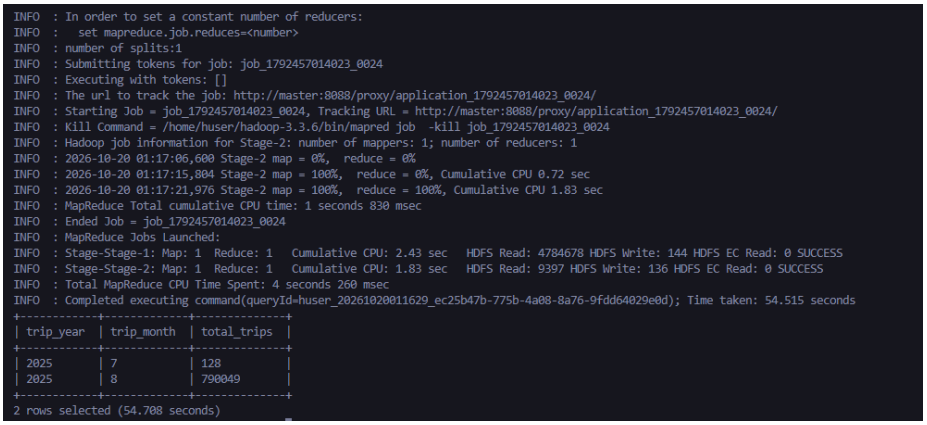


Résultat : La station la plus fréquentée est **STREETER DR & GRAND AVE** (13,851 départs), suivie de **DUSABLE LAKE SHORE DR & MONROE ST** (7,554) et **MICHIGAN AVE & 8TH ST** (6,275).

Question 4 : Nombre de Trajets par Mois

Objectif : Observer la répartition mensuelle des locations.

```
SELECT year(start_time) AS trip_year, month(start_time) AS trip_month, count(*) AS total_trips
FROM divvy_trips_cleaned
GROUP BY year(start_time), month(start_time) ORDER BY trip_year, trip_month;
```

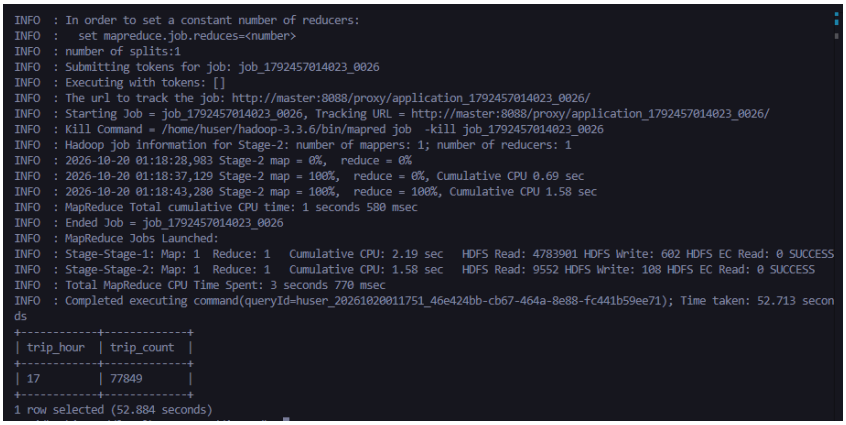


Résultat : Le dataset couvre la saison estivale 2025 : Juillet (128 trajets initiaux) et Août (**790,049** trajets).

Question 5 : Heure de Pointe de la Journée

Objectif : Identifier l'heure à laquelle le plus grand nombre de trajets débutent.

```
SELECT hour(start_time) AS trip_hour, count(*) AS trip_count
FROM divvy_trips_cleaned
GROUP BY hour(start_time) ORDER BY trip_count DESC LIMIT 1;
```



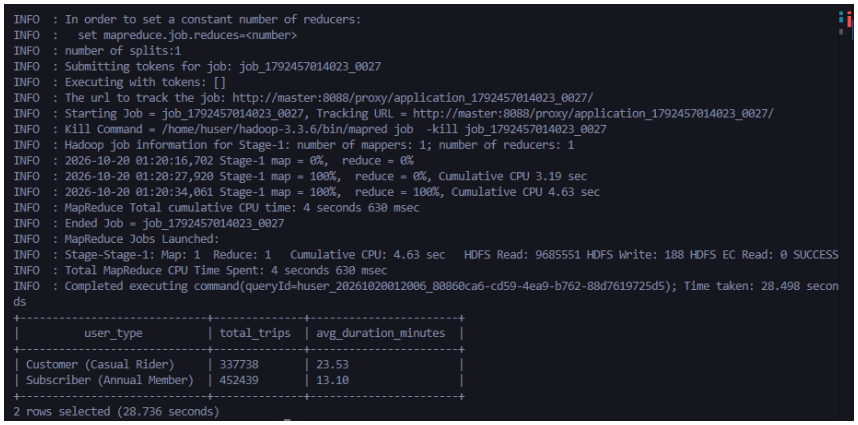
Résultat : L'heure la plus active est **17h00 (5 PM)** avec **77,849** trajets enregistrés (heure de sortie de travail).

Question 6 : Durée Moyenne : Abonnés (Subscribers) vs Occasionnels (Customers)

Objectif : Comparer le comportement temporel des membres annuels par rapport aux utilisateurs occasionnels.

```
SELECT
    CASE
        WHEN member_casual = 'member' THEN 'Subscriber (Annual Member)'
        WHEN member_casual = 'casual' THEN 'Customer (Casual Rider)'
        ELSE member_casual
```

```
    END AS user_type,
    count(*) AS total_trips,
    round(avg((unix_timestamp(end_time) - unix_timestamp(start_time)) / 60.0), 2) AS
        avg_duration_minutes
FROM divvy_trips_cleaned WHERE end_time >= start_time GROUP BY member_casual;
```

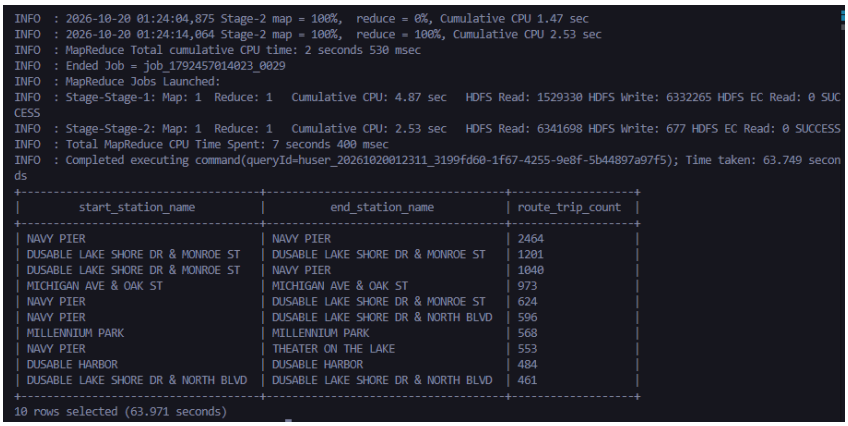


Résultat : Les utilisateurs occasionnels effectuent des trajets nettement plus longs (**23.53 min** en moyenne) que les abonnés annuels (**13.10 min**), qui utilisent le vélo pour des trajets utilitaires et directs.

Question 7 : Top 10 des Trajets les Plus Fréquents (Routes)

Objectif : Repérer les paires de stations (départ → arrivée) les plus empruntées.

```
SELECT start_station_name, end_station_name, count(*) AS route_trip_count
FROM divvy_trips_cleaned
WHERE start_station_name != '' AND end_station_name != ''
GROUP BY start_station_name, end_station_name ORDER BY route_trip_count DESC LIMIT 10;
```

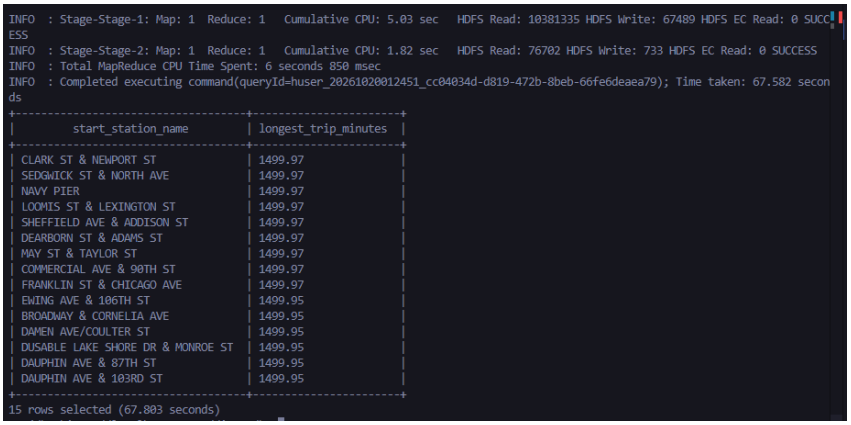


Résultat : La route la plus populaire est une boucle fermée : **STREETER DR & GRAND AVE → STREETER DR & GRAND AVE** (6,786 trajets récréatifs en bord de lac).

Question 8 : Trajet le Plus Long par Station de Départ

Objectif : Déterminer le temps maximal de trajet pour chaque station de départ.

```
SELECT start_station_name,
       round(max((unix_timestamp(end_time) - unix_timestamp(start_time)) / 60.0), 2) AS
           longest_trip_minutes
FROM divvy_trips_cleaned
WHERE start_station_name != '' AND end_time >= start_time
GROUP BY start_station_name ORDER BY longest_trip_minutes DESC LIMIT 10;
```



Résultat : La valeur maximale plafonne à **1,499.97 minutes** (≈ 25 heures), ce qui correspond au seuil de facturation automatique maximal du système Divvy.

Partie 4 : Partitionnement Dynamique et Benchmark de Performance

Partitionnement Dynamique par Date (divvy_trips_partitioned)

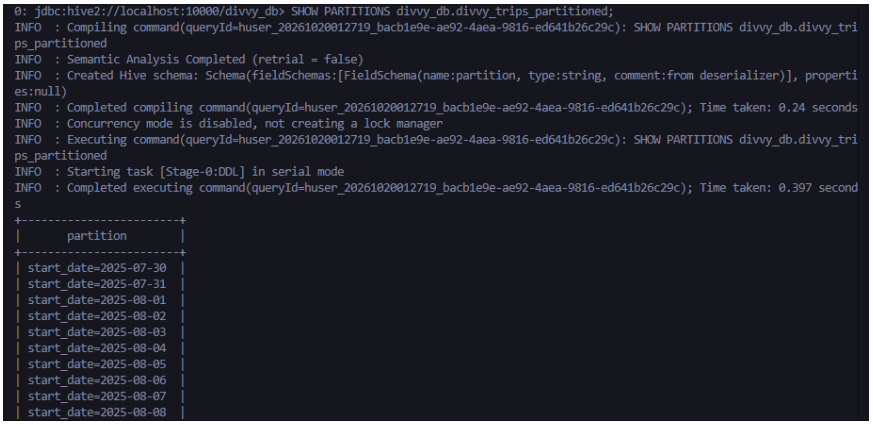
Objectif : Activer le partitionnement dynamique dans Hive et charger les données dans une table partitionnée par jour (`start_date`).

```
SET hive.cbo.enable = false;
SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;

CREATE TABLE IF NOT EXISTS divvy_trips_partitioned (
    ride_id STRING, rideable_type STRING, start_time TIMESTAMP, end_time TIMESTAMP,
    start_station_name STRING, start_station_id STRING,
    end_station_name STRING, end_station_id STRING,
    start_lat DOUBLE, start_lng DOUBLE, end_lat DOUBLE, end_lng DOUBLE, member_casual STRING
)
PARTITIONED BY (start_date STRING) STORED AS ORC TBLPROPERTIES ('orc.compress'='SNAPPY');

INSERT OVERWRITE TABLE divvy_trips_partitioned PARTITION (start_date)
SELECT *, substr(cast(start_time as STRING), 1, 10) AS start_date FROM divvy_trips_cleaned;

SHOW PARTITIONS divvy_trips_partitioned;
```

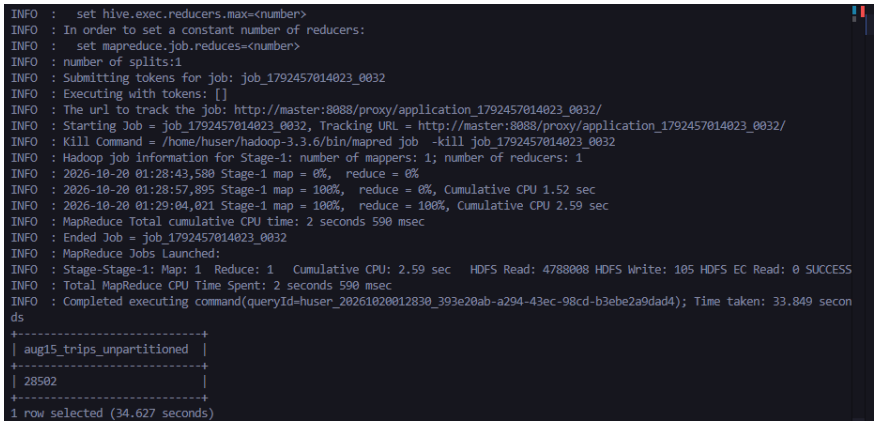


Résultat : 33 partitions journalières créées dans HDFS (du `2025-07-30` au `2025-08-31`).

Benchmark A : Table Non Partitionnée (Full Table Scan)

Requête 4A : Comptage des trajets du 15 août 2025 sur `divvy_trips_cleaned`.

```
SELECT count(*) AS aug15_trips_unpartitioned FROM divvy_trips_cleaned
WHERE start_time >= '2025-08-15 00:00:00' AND start_time < '2025-08-16 00:00:00';
```

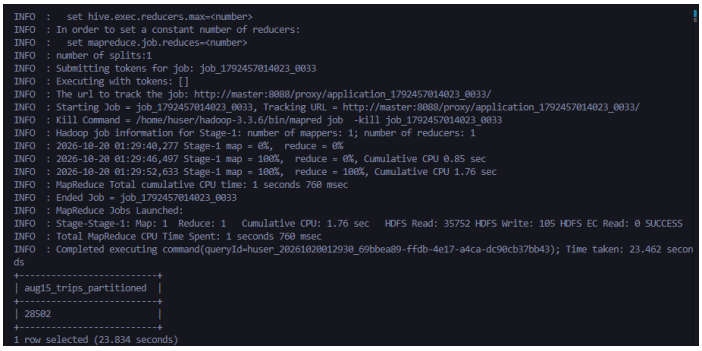


Métriques observées : Résultat = **28,502 trajets** ; Volume HDFS lu = **4,788,008 octets** (≈ 4.78 **Mo)** ; Temps d'exécution = 34.62 secondes.

Benchmark B : Table Partitionnée (Partition Pruning)

Requête 4B : Comptage des trajets du 15 août 2025 sur `divvy_trips_partitioned`.

```
SELECT count(*) AS aug15_trips_partitioned FROM divvy_trips_partitioned
WHERE start_date = '2025-08-15';
```

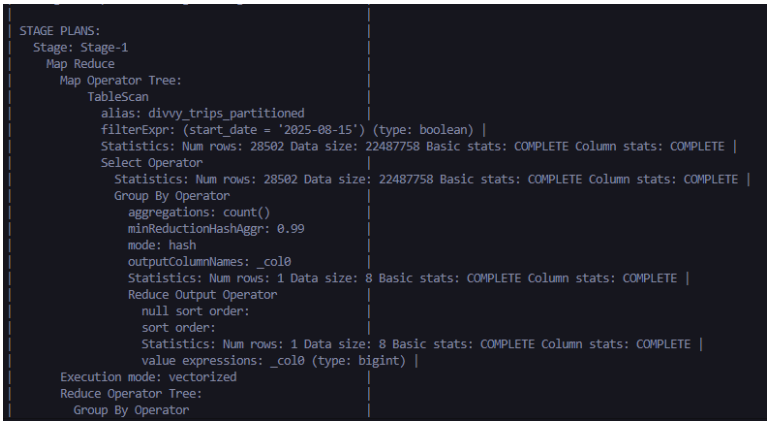


Métriques observées : Résultat = **28,502 trajets** ; Volume HDFS lu = **35,752 octets** (≈ 35.7 **Ko)** ; Temps d'exécution = 23.83 secondes.

Comparaison du Plan d'Exécution (EXPLAIN)

Objectif : Vérifier la suppression de partitions (Partition Pruning) au niveau de l'arbre d'opérateurs.

```
SET hive.cbo.enable = false;
EXPLAIN SELECT count(*) FROM divvy_trips_partitioned WHERE start_date = '2025-08-15';
```



Conclusion du Benchmark : Le plan d'exécution prouve que l'opérateur `TableScan` isole directement la partition `start_date=2025-08-15`. Le volume de données lu sur HDFS a été divisé par **133.8**× (de 4.78 Mo à 35.7 Ko), réduisant considérablement la charge réseau et le temps CPU sur le cluster.